

WebAssembly Benchmarking

Carly Venenciano, Occidental College

December 9, 2023

Abstract

This paper discusses our ongoing research on WebAssembly (Wasm) performance across diverse virtual machines (VMs). We analyze three timed metrics across different VM configurations and versions. Building upon Dr. Ben L. Titzer’s research and his Wizard Engine—a VM dedicated to Wasm research, our study aims to contribute insights into Wasm performance tradeoffs, striving to assist users in comprehensively understanding the differences between various Wasm engines and configurations.

Introduction

WebAssembly, commonly referred to as Wasm, serves as a portable compiler target for compiling various programming languages into bytecode to be run on an operating system. Wasm’s efficiency makes it invaluable for performance-intensive projects such as games, music streaming, and video editing (Yegulalp 2022). Recognizing its potential, widely-used web services like Google Earth and Figma have already integrated Wasm because of its near-native speed (Yegulalp 2022). Moreover, Wasm’s versatility and speed makes it an ideal solution for a range of applications.

Our research focus lies in benchmarking common web and non-web Wasm engines—virtual machines designed to execute programs in a specific language on a host machine. Within each engine, we examine tiering configurations, involving interpreters and compilers. Interpreters decode and execute each byte of bytecode individually, while compilers translate bytecode into native code for execution. It’s worth noting that some engines contain multiple interpreters and compilers while other engines contain only one.

Thus far, we’ve collected data on ten engines with a total of 18 engine configurations listed in

Table 1. This diverse set of engines represents a broad spectrum of Wasm implementations, ranging from web browsers to non-web environments. By scrutinizing their performance characteristics, we aim to shed light on the intricate trade-offs and nuances involved in selecting the most suitable Wasm engine for varied use cases.

Table 1: Wasm Engine Configurations

Engine	Configuration	Details
JavaScriptCore (Safari)	<code>jsc-int</code>	JITs disabled, rewriting interpreter enabled
	<code>jsc-bbq</code>	Interpreter enabled, only BBQ quick JIT enabled
	<code>jsc-omg</code>	Interpreter enabled, only OMG optimizing JIT enabled
SpiderMonkey (Firefox)	<code>sm-base</code>	Only baseline compiler enabled
	<code>sm-opt</code>	Only optimizing compiler enabled
V8 (Chrome)	<code>v8-liftoff</code>	Only Liftoff baseline JIT enabled
	<code>v8-turbonfan</code>	Only TurboFan optimizing JIT enabled
Wizard (non-web engine)	<code>wizeng-int</code>	Only interpreter enabled
	<code>wizeng-jit</code>	Only JIT compiler enabled
iwasm (non-web)	<code>iwasm-int</code>	Only interpreter enabled
	<code>iwasm-fjit</code>	Only fast JIT enabled
Wasmer (non-web)	<code>wasmer</code>	Default with Cranelift
	<code>wasmer-base</code>	Single-pass compiler enabled
wasmtime (non-web)	Default	Cranelift optimizing compiler, written in Rust
Wasm3 (non-web)	Default	Interpreter for embedded systems
WAVM (non-web)	Default	LLVM ahead-of-time compilation
wazero (non-web)	Default	Written in Go

Methodology

We evaluate Wasm engine performance by the execution times of line items from three benchmark suites. Line items are individual programs of simple mathematical operations like scalar multiplication or differentiation. Our three benchmark suites are PolyBench/C, Ostrich, and Libsodium. Each suit represents a set of interrelated programs. We tested on

29 line items from PolyBench/C, 11 from Ostrich, and 39 from Libsodium, resulting in a total of 79 line items. We chose these benchmark suites because they encompass common numerical computations prevalent in a variety of programs.

Our three timed metrics for experimentation are **total time**, **main (execution) time**, and **set up time**. **Total time** signifies the entire process, encompassing the duration from engine startup to program completion. **Main time** represents the duration it takes for the program to execute its main function. **Set up time** accounts for program-related startup costs, including tasks such as loading, parsing, validating, and potentially compiling Wasm modules. This metric extends up to the execution of the first bytecode.

Each line item corresponds to its own Wasm file, executed through a function named `btime`. To collect timed metrics, we employed the `btime` function, measuring in seconds. Two types of text files were generated: one capturing total time (*T*-files), and another measuring setup and startup time (*S*-files). For these experiments we did not measure startup time and used values gathered from Dr.Titzer’s previous work.

When creating our *T*-files, `btime` ran each Wasm file on every engine-configuration ten times and recorded the raw data, engine version, and engine’s build date. Our *S*-files have the same format but we ran `btime` with altered Wasm files that were coded to exit the program right before the engine was about to execute the compiled bytecode. Using this data we calculated our three metrics as follows.

$$\text{Total time} = T\text{-files}$$

$$\text{Main time} = T\text{-files} - S\text{-files}$$

$$\text{Set up time} = S\text{-files} - \text{Startup time}$$

In addition to these metrics, we calculated averages, the 5th percentile, the 95th percentile, minimum, and maximum times from the raw data.

Data was primarily collected on a 64-bit Linux, a popular operating system, and limited

data was collected on an EC2 t2.micro Instance, for convenience reasons. Our results were uploaded to a PostgreSQL database with more than 2100 lines of data and 18 columns of metrics.

Results & Extensions

Our results show the tradeoff between the set up time and main time. In alignment with Dr.Titzer’s results, our findings show that engines with quicker set up time tend to have a longer main time and vice versa (Titzer 2023). We expect that greater data collection will emphasize this trend. Additionally, we are curious to see how this tradeoff changes with different engine versions. Thus, to extend our research we are focusing on gathering data on older engine versions.

Furthermore, we are tweaking our methodology. Due to measurement error, there are some instances in which the *T*-files’s average is greater than the *S*-file’s average. As a result, for very short-running line items, the main time is calculated to be a negative value. The statistical variance of engine startup is the source of these issues, however, results from these experiments may vary when executed on a different machine. For negative time calculations, we recorded such data as zero seconds. Moving forward, we aim to enhance the robustness of our experiments by increasing the number of runs per experiment. This proactive measure is intended to mitigate the likelihood of encountering negative time and to yield more precise running time averages as the project progresses.

In addition to our experimentation, we created an API server and web page to visualize our results. The purpose of this webpage is to be a useful resource for new and existing Wasm users. As we continue our research, we plan to generate more dynamic and interactive graphs for users to compare our results in numerous ways. Furthermore, we want to allow users to parse our database for specific raw data.

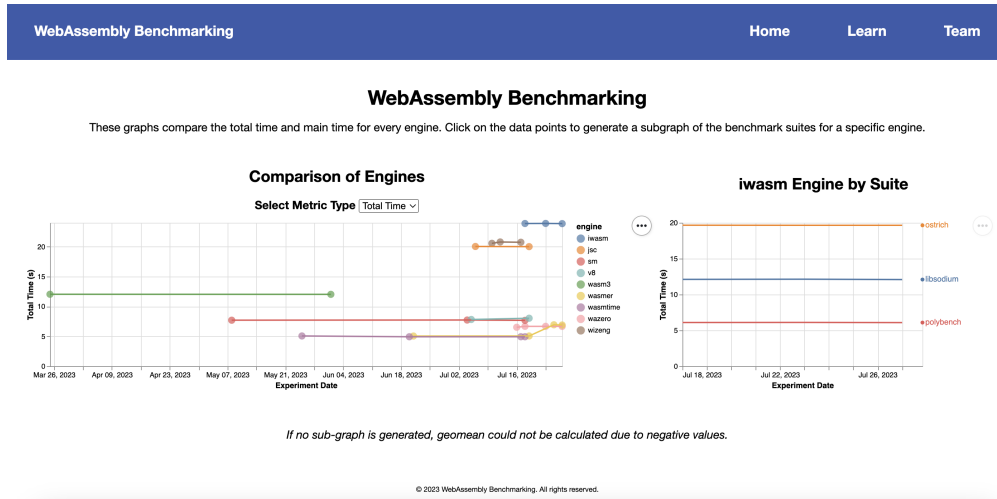


Figure 1: Landing Page of Website

Conclusion

In the course of this project, we conducted a comprehensive examination of Wasm engine performance across three types of speeds. Our findings reveal an emerging pattern suggesting that engines boasting faster compilation times tend to exhibit slower execution times, and conversely, those with slower compilation times demonstrate faster execution. As we continue to accumulate additional data and develop more comprehensive visualizations, these observed tradeoffs will become more evident and valuable for the broader Wasm community.

Furthermore, the recent establishment of the WebAssembly Benchmarking Center signifies the beginning of a dedicated benchmarking endeavor. Through collaborative efforts with other groups, our aim is to provide Wasm users with increasingly insightful data. Looking ahead, our ambitions include expanding our testing to encompass a broader array of machines, gathering additional historical data, and exploring potential collaboration with other benchmarking initiatives. This ongoing commitment underscores our dedication to advancing the understanding of Wasm engine performance for the benefit of the entire user community.

Bibliography

Titzer, Ben L. “Whose Baseline Compiler Is It Anyway?”, arXiv preprint. *arXiv*, arxiv.org/abs/2305.13241, Nov. 2023, <https://doi.org/10.48550/arXiv.2305.13241>.

Yegulalp, Serdar. “What Is WebAssembly? The next-Generation Web Platform Explained”. *Info World*, May 2022. www.infoworld.com/article/3291780/what-is-webassembly-the-next-generation-web-platform-explained.html.